第13章 字 符 串

可以证明，字符串操作是计算机程序设计中最常见的行为。

尤其是在Java大展拳脚的Web系统中更是如此。在本章中，我们将深入学习在Java语言中应用最广泛的**String**类，并研究与之相关的类及工具。

13.1 不可变String

String对象是不可变的。查看JDK文档你就会发现，**String**类中每一个看起来会修改**String**值的方法，实际上都是创建了一个全新的**String**对象，以包含修改后的字符串内容。而最初的**String**对象则丝毫未动。

看看下面的代码：

```
//: strings/Immutable.java
import static net.mindview.util.Print.*;

public class Immutable {
  public static String upcase(String s) {
    return s.toUpperCase();
  }
  public static void main(String[] args) {
    String q = "howdy";
    print(q); // howdy
    String qq = upcase(q);
    print(qq); // HOWDY
    print(q); // howdy
  }
} /* Output:
howdy
HOWDY
howdy
*///:~
```


当把**q**传给**upcase()**方法时，实际传递的是引用的一个拷贝。其实，每当把**String**对象作为方法的参数时，都会复制一份引用，而该引用所指的对象其实一直待在单一的物理位置上，从未动过。

回到**upcase()**的定义，传入其中的引用有了名字**s**，只有**upcase()**运行的时候，局部引用**s**才存在。一旦**upcase()**运行结束，**s**就消失了。当然了，**upcase()**的返回值，其实只是最终结果的引用。这足以说明，**upcase()**返回的引用已经指向了一个新的对象，而原本的**q**则还在原地。

String的这种行为方式其实正是我们想要的。例如：

```
String s = "asdf";
String x = Immutable.upcase(s);
```

难道你真的希望**upcase()**改变其参数吗？对于一个方法而言，参数是为该方法提供信息的，而不是想让该方法改变自己的。在阅读这段代码时，读者自然就会有这样的感觉。这一点很重要，正是有了这种保障，才使得代码易于编写与阅读。

13.2 重载“+”与StringBuilder

String对象是不可变的，你可以给一个**String**对象加任意多的别名。因为**String**对象具有

只读特性，所以指向它的任何引用都不可能改变它的值，因此，也就不会对其他的引用有什么影响。

不可变性会带来一定的效率问题。为**String**对象重载的“+”操作符就是一个例子。重载的意思是，一个操作符在应用于特定的类时，被赋予了特殊的意义（用于**String**的“+”与“+=”是Java中仅有的两个重载过的操作符，而Java并不允许程序员重载任何操作符[⊖]）。

504

操作符“+”可以用来连接**String**：

```
//: strings/Concatenation.java

public class Concatenation {
  public static void main(String[] args) {
    String mango = "mango";
    String s = "abc" + mango + "def" + 47;
    System.out.println(s);
  }
} /* Output:
abcmangodef47
*///:~
```

可以想象一下，这段代码可能是这样工作的：**String**可能有一个**append()**方法，它会生成一个新的**String**对象，以包含“abc”与**mango**连接后的字符串。然后，该对象再与“def”相连，生成另一个新的**String**对象，依此类推。

这种工作方式当然也行得通，但是为了生成最终的**String**，此方式会产生一大堆需要垃圾回收的中间对象。我猜想，Java设计师一开始就是这么做的（这也是软件设计中的一个教训：除非你用代码将系统实现，并让它动起来，否则你无法真正了解它会有什么问题），然后他们发现其性能相当糟糕。

想看看以上代码到底是如何工作的吗，可以用JDK自带的工具**javap**来反编译以上代码。命令如下：

```
javap -c Concatenation
```

这里的**-c**标志表示将生成JVM字节码。我剔除掉了不感兴趣的部分，然后作了一点点修改，于是有了以下的字节码：

505

```
public static void main(java.lang.String[]);
  Code:
   Stack=2, Locals=3, Args_size=1
   0:    ldc #2; //String mango
   2:    astore_1
   3:    new #3; //class StringBuilder
   6:    dup
   7:    invokespecial #4; //StringBuilder."<init>":()
   10:   ldc #5; //String abc
   12:   invokevirtual #6; //StringBuilder.append:(String)
   15:   aload_1
   16:   invokevirtual #6; //StringBuilder.append:(String)
   19:   ldc #7; //String def
   21:   invokevirtual #6; //StringBuilder.append:(String)
   24:   bipush 47
   26:   invokevirtual #8; //StringBuilder.append:(I)
   29:   invokevirtual #9; //StringBuilder.toString:()
```

⊖ C++允许程序员任意重载操作符。由于这通常是很复杂的过程（参见《C++编程思想（第2版）》第10章——本书中英文版均已由机械工业出版社出版），所以Java设计者认为这是“糟糕的”功能，不应该包括在Java中。其实重载操作符并没有糟糕到只能让它们自己去重载的地步，但具有讽刺意味的是，与C++相比，在Java中使用操作符重载要容易得多。这一点可以在Python（参考www.Python.org）与C#中看到，它们都具有垃圾回收与简单易懂的操作符重载机制。

```
32:   astore_2
33:   getstatic #10; //Field System.out:PrintStream;
36:   aload_2
37:   invokevirtual #11; // PrintStream.println:(String)
40:   return
```

如果你有汇编语言的经验，以上代码一定看着眼熟，其中的**dup**与**invokevirtural**语句相当于Java虚拟机上的汇编语句。即使你完全不了解汇编语言也无须担心，需要注意的重点是：编译器自动引入了**java.lang.StringBuilder**类。虽然我们在源代码中并没有使用 **StringBuilder**类，但是编译器却自作主张地使用了它，因为它更高效。

在这个例子中，编译器创建了一个**StringBuilder**对象，用以构造最终的**String**，并为每个字符串调用一次**StringBuilder**的**append()**方法，总计四次。最后调用**toString()**生成结果，并存为**s**（使用的命令为**astore_2**）。

现在，也许你会觉得可以随意使用**String**对象，反正编译器会为你自动地优化性能。可是在这之前，让我们更深入地看看编译器能为我们优化到什么程度。下面的程序采用两种方式生成一个**String**：方法一使用了多个**String**对象；方法二在代码中使用了**StringBuilder**。

```
//: strings/WhitherStringBuilder.java

public class WhitherStringBuilder {
  public String implicit(String[] fields) {
    String result = "";
    for(int i = 0; i < fields.length; i++)
      result += fields[i];
    return result;
  }
  public String explicit(String[] fields) {
    StringBuilder result = new StringBuilder();
    for(int i = 0; i < fields.length; i++)
      result.append(fields[i]);
    return result.toString();
  }
} ///:~
```

506

现在运行**javap -c WitherStringBuilder**，可以看到两个方法对应的（简化过的）字节码。首先是**implicit()**方法：

```
public java.lang.String implicit(java.lang.String[]);
  Code:
   0:    ldc #2; //String
   2:    astore_2
   3:    iconst_0
   4:    istore_3
   5:    iload_3
   6:    aload_1
   7:    arraylength
   8:    if_icmpge 38
   11:   new #3; //class StringBuilder
   14:   dup
   15:   invokespecial #4; // StringBuilder."<init>":()
   18:   aload_2
   19:   invokevirtual #5; // StringBuilder.append:()
   22:   aload_1
   23:   iload_3
   24:   aaload
   25:   invokevirtual #5; // StringBuilder.append:()
   28:   invokevirtual #6; // StringBuilder.toString:()
   31:   astore_2
   32:   iinc 3, 1
   35:   goto 5
   38:   aload_2
   39:   areturn
```

注意从第8行到第35行构成了一个循环体。第8行：对堆栈中的操作数进行“大于或等于的整数比较运算”，循环结束时跳到第38行。第35行：返回循环体的起始点（第5行）。要注意的重点是：**StringBuilder**是在循环之内构造的，这意味着每经过循环一次，就会创建一个新的**StringBuilder**对象。

下面是**explicit()**方法对应的字节码：

```
public java.lang.String explicit(java.lang.String[]);
  Code:
   0:    new #3; //class StringBuilder
   3:    dup
   4:    invokespecial #4; // StringBuilder."<init>":()
   7:    astore_2
   8:    iconst_0
   9:    istore_3
   10:   iload_3
   11:   aload_1
   12:   arraylength
   13:   if_icmpge 30
   16:   aload_2
   17:   aload_1
   18:   iload_3
   19:   aaload
   20:   invokevirtual #5; // StringBuilder.append:()
   23:   pop
   24:   iinc 3, 1
   27:   goto 10
   30:   aload_2
   31:   invokevirtual #6; // StringBuilder.toString:()
   34:   areturn
```

507

可以看到，不仅循环部分的代码更简短、更简单，而且它只生成了一个**StringBuilder**对象。显式地创建**StringBuilder**还允许你预先为其指定大小。如果你已经知道最终的字符串大概有多长，那预先指定**StringBuilder**的大小可以避免多次重新分配缓冲。

因此，当你为一个类编写**toString()**方法时，如果字符串操作比较简单，那就可以信赖编译器，它会为你合理地构造最终的字符串结果。但是，如果你要在**toString()**方法中使用循环，那么最好自己创建一个**StringBuilder**对象，用它来构造最终的结果。请参考以下示例：

```
//: strings/UsingStringBuilder.java
import java.util.*;

public class UsingStringBuilder {
  public static Random rand = new Random(47);
  public String toString() {
    StringBuilder result = new StringBuilder("[");
    for(int i = 0; i < 25; i++) {
      result.append(rand.nextInt(100));
      result.append(", ");
    }
    result.delete(result.length()-2, result.length());
    result.append("]");
    return result.toString();
  }
  public static void main(String[] args) {
    UsingStringBuilder usb = new UsingStringBuilder();
    System.out.println(usb);
  }
} /* Output:
[58, 55, 93, 61, 61, 29, 68, 0, 22, 7, 88, 28, 51, 89, 9,
78, 98, 61, 20, 58, 16, 40, 11, 22, 4]
*///:~
```

508

最终的结果是用**append()**语句一点点拼接起来的。如果你想走捷径，例如**append(a+ “:” +c)**，

那编译器就会掉入陷阱，从而为你另外创建一个**StringBuilder**对象处理括号内的字符串操作。

如果拿不准该用哪种方式，随时可以用**javap**来分析你的程序。

StringBuilder提供了丰富而全面的方法，包括**insert()**、**repleace()**、**substring()**甚至**reverse()**，但是最常用的还是**append()**和**toString()**。还有**delete()**方法，上面的例子中我们用它删除最后一个逗号与空格，以便添加右括号。

StringBuilder是Java SE5引入的，在这之前Java用的是**StringBuffer**。后者是线程安全的（参见第21章），因此开销也会大些，所以在Java SE5/6中，字符串操作应该还会更快一点。

练习1：(2) 分析**reusing/SprinklerSystem.java**的**SprinklerSystem.tString()**方法，看看明确地使用**StringBuilder**是否确实能够避免产生过多的**StringBuilder**对象。

13.3 无意识的递归

Java中的每个类从根本上都是继承自**Object**，标准容器类自然也不例外。因此容器类都有**toString()**方法，并且覆写了该方法，使得它生成的**String**结果能够表达容器自身，以及容器所包含的对象。例如**ArrayList.toString()**，它会遍历**ArrayList**中包含的所有对象，调用每个元素上的**toString()**方法：

```
//: strings/ArrayListDisplay.java
import generics.coffee.*;
import java.util.*;
public class ArrayListDisplay {
  public static void main(String[] args) {
    ArrayList<Coffee> coffees = new ArrayList<Coffee>();
    for(Coffee c : new CoffeeGenerator(10))
      coffees.add(c);
    System.out.println(coffees);
  }
} /* Output:
[Americano 0, Latte 1, Americano 2, Mocha 3, Mocha 4, Breve
5, Americano 6, Latte 7, Cappuccino 8, Cappuccino 9]
*///:~
```

509

如果你希望**toString()**方法打印出对象的内存地址，也许你会考虑使用**this**关键字：

```
//: strings/InfiniteRecursion.java
// Accidental recursion.
// {RunByHand}
import java.util.*;

public class InfiniteRecursion {
  public String toString() {
    return " InfiniteRecursion address: " + this + "\n";
  }
  public static void main(String[] args) {
    List<InfiniteRecursion> v =
      new ArrayList<InfiniteRecursion>();
    for(int i = 0; i < 10; i++)
      v.add(new InfiniteRecursion());
    System.out.println(v);
  }
} ///:~
```

当你创建了**InfiniteRecursion**对象，并将其打印出来的时候，你会得到一串非常长的异常。如果你将该**InfiniteRecursion**对象存入一个**ArrayList**中，然后打印该**ArrayList**，你也会得到同样的异常。其实，当如下代码运行时：

```
"InfiniteRecursion address: " + this
```

这里发生了自动类型转换，由**InfiniteRecursion**类型转换成**String**类型。因为编译器看到一个**String**对象后面跟着一个“+”，而再后面的对象不是**String**，于是编译器试着将this转换成一个**String**。它怎么转换呢，正是通过调用**this**上的**toString()**方法，于是就发生了递归调用。

510

如果你真的想要打印出对象的内存地址，应该调用**Object.toString()**方法，这才是负责此任务的方法。所以，你不该使用**this**，而是应该调用**super.toString()**方法。

练习2：(1) 修复**InfiniteRecursion.java**。

13.4 String上的操作

以下是**String**对象具备的一些基本方法。重载的方法归纳在同一行中：

方　法	参数，重载版本	应　用
构造器	重载版本：默认版本，**String**，**StringBuilder**，**StringBuffer**，**char**数组，**byte**数组	创建**String**对象
length()		**String**中字符的个数
charAt()	**Int**索引	取得**String**中该索引位置上的**char**
getChars(), **getBytes()**	要复制部分的起点和终点的索引，复制的目标数组，目标数组的起始索引	复制**char**或**byte**到一个目标数组中
toCharArray()		生成一个**char[]**，包含**String**的所有字符
equals(), **equalsIgnoreCase()**	与之进行比较的**String**	比较两个**String**的内容是否相同
compareTo()	与之进行比较的**String**	按词典顺序比较**String**的内容，比较结果为负数、零或正数。注意，大小写并不等价
contains()	要搜索的**CharSequence**	如果该**String**对象包含参数的内容，则返回**true**
contentEquals()	与之进行比较的**CharSequence**或**StringBuffer**	如果该**String**与参数的内容完全一致，则返回**true**
equalsIgnoreCase()	与之进行比较的**String**	忽略大小写，如果两个**String**的内容相同，则返回**true**
regionMatcher()	该**String**的索引偏移量，另一个**String**及其索引偏移量，要比较的长度。重载版本增加了“忽略大小写”功能	返回**boolean**结果，以表明所比较区域是否相等
startsWith()	可能的起始**String**。重载版本在参数中增加了偏移量	返回**boolean**结果，以表明该**String**是否以此参数起始
endsWith()	该**String**可能的后缀**String**	返回**boolean**结果，以表明此参数是否该字符串的后缀
indexOf(), **lastIndexOf()**	重载版本包括：**char**，**char**与起始索引，**String**，**String**与起始索引	如果该**String**并不包含此参数，就返回-1；否则返回此参数在**String**中的起始索引。**lastIndexOf()**是从后向前搜索
substring()（**subSequence()**）	重载版本：起始索引；起始索引+终点坐标	返回一个新的**String**，以包含参数指定的子字符串

511

(续)

方 法	参数，重载版本	应 用
concat()	要连接的String	返回一个新的String对象，内容为原始String连接上参数String
replace()	要替换掉的字符，用来进行替换的新字符。也可以用一个CharSequence来替换另一个CharSequence	返回替换字符后的新String对象。如果没有替换发生，则返回原始的String对象
toLowerCase() toUpperCase()		将字符的大小写改变后，返回一个新String对象。如果没有改变发生，则返回原始的String对象
trim()		将String两端的空白字符删除后，返回一个新的String对象。如果没有改变发生，则返回原始的String对象
valueOf()	重载版本：Object；char[]；char[]，偏移量，与字符个数；boolean；char；int；long；float；double	返回一个表示参数内容的String
intern()		为每个唯一的字符序列生成一个且仅生成一个String引用

512

从这个表中可以看出，当需要改变字符串的内容时，String类的方法都会返回一个新的String对象。同时，如果内容没有发生改变，String的方法只是返回指向原对象的引用而已。这可以节约存储空间以及避免额外的开销。

本章稍后还将介绍正则表达式在String方法中的应用。

513

13.5 格式化输出

在长久的等待之后，Java SE5终于推出了C语言中printf()风格的格式化输出这一功能。这不仅使得控制输出的代码更加简单，同时也给与Java开发者对于输出格式与排列更强大的控制能力[⊖]。

13.5.1 printf()

C语言中的printf()并不能像Java那样连接字符串，它使用一个简单的格式化字符串，加上要插入其中的值，然后将其格式化输出。printf()并不使用重载的“+”操作符（C没有重载）来连接引号内的字符串或字符串变量，而是使用特殊的占位符来表示数据将来的位置。而且它还将插入格式化字符串的参数，以逗号分隔，排成一行。

例如：

```
printf("Row 1: [%d %f]\n", x, y);
```

这一行代码在运行的时候，首先将x的值插入到%d的位置，然后将y的值插入到%f的位置。这些占位符称作格式修饰符，它们不但说明了插入数据的位置，同时还说明了将插入什么类型的变量，以及如何对其格式化。在这个例子中，%d表示x是一个整数，%f表示y是以一个浮点数（float或者double）。

13.5.2 System.out.format()

Java SE5引入的format方法可用于PrintStream或PrintWriter对象（我们将在第18章学习它

⊖ Mark Welsh帮助我撰写了本节以及13.7节。

们)，其中也包括**System.out**对象。**format()**方法模仿自C的**printf()**。如果你比较怀旧的话，也可以使用**printf()**。以下是一个简单的示例：

```
//: strings/SimpleFormat.java

public class SimpleFormat {
  public static void main(String[] args) {
    int x = 5;
    double y = 5.332542;
    // The old way:
    System.out.println("Row 1: [" + x + " " + y + "]");
    // The new way:
    System.out.format("Row 1: [%d %f]\n", x, y);
    // or
    System.out.printf("Row 1: [%d %f]\n", x, y);
  }
} /* Output:
Row 1: [5 5.332542]
Row 1: [5 5.332542]
Row 1: [5 5.332542]
*///:~
```

514

可以看到，**format()**与**printf()**是等价的，它们只需要一个简单的格式化字符串，加上一串参数即可，每个参数对应一个格式修饰符。

13.5.3 Formatter类

在Java中，所有新的格式化功能都由**java.util.Formatter**类处理。可以将**Formatter**看作一个翻译器，它将你的格式化字符串与数据翻译成需要的结果。当你创建一个**Formatter**对象的时候，需要向其构造器传递一些信息，告诉它最终的结果将向哪里输出：

```
//: strings/Turtle.java
import java.io.*;
import java.util.*;

public class Turtle {
  private String name;
  private Formatter f;
  public Turtle(String name, Formatter f) {
    this.name = name;
    this.f = f;
  }
  public void move(int x, int y) {
    f.format("%s The Turtle is at (%d,%d)\n", name, x, y);
  }
  public static void main(String[] args) {
    PrintStream outAlias = System.out;
    Turtle tommy = new Turtle("Tommy",
      new Formatter(System.out));
    Turtle terry = new Turtle("Terry",
      new Formatter(outAlias));
    tommy.move(0,0);
    terry.move(4,8);
    tommy.move(3,4);
    terry.move(2,5);
    tommy.move(3,3);
    terry.move(3,3);
  }
} /* Output:
Tommy The Turtle is at (0,0)
Terry The Turtle is at (4,8)
Tommy The Turtle is at (3,4)
Terry The Turtle is at (2,5)
Tommy The Turtle is at (3,3)
Terry The Turtle is at (3,3)
*///:~
```

515

所有的**tommy**都将输出到**System.out**，而所有的**terry**则都输出到**System.out**的一个别名中。**Formatter**的构造器经过重载可以接受多种输出目的地，不过最常用的还是**PrintStream()**（如上例）、**OutputStream**和**File**。在第18章中你将看到与之相关的更多信息。

练习3：(1) 修改**Turtle.java**，使之将结果输出到**System.err**中。

前面的示例中还使用了一个新的格式化修饰符**%s**，它表示插入的参数是**String**类型。这个例子使用的是最简单类型的格式修饰符——它只具有转换类型而没有其他功能。

13.5.4 格式化说明符

在插入数据时，如果想要控制空格与对齐，你需要更精细复杂的格式修饰符。以下是其抽象的语法：

```
%[argument_index$][flags][width][.precision]conversion
```

最常见的应用是控制一个域的最小尺寸，这可以通过指定**width**来实现。**Formatter**对象通过在必要时添加空格，来确保一个域至少达到某个长度。在默认的情况下，数据是右对齐，不过可以通过使用“-”标志来改变对齐方向。

516

与**width**相对的是**precision**，它用来指明最大尺寸。**width**可以应用于各种类型的数据转换，并且其行为方式都一样。**precision**则不然，不是所有类型的数据都能使用**precision**，而且，应用于不同类型的数据转换时，**precision**的意义也不同。在将**precision**应用于**String**时，它表示打印**String**时输出字符的最大数量。而在将**precision**应用于浮点数时，它表示小数部分要显示出来的位数（默认是6位小数），如果小数位数过多则舍入，太少则在尾部补零。由于整数没有小数部分，所以**precision**无法应用于整数，如果你对整数应用**precision**，则会触发异常。

下面的程序应用格式修饰符来打印一个购物收据：

```
//: strings/Receipt.java
import java.util.*;

public class Receipt {
  private double total = 0;
  private Formatter f = new Formatter(System.out);
  public void printTitle() {
    f.format("%-15s %5s %10s\n", "Item", "Qty", "Price");
    f.format("%-15s %5s %10s\n", "----", "---", "-----");
  }
  public void print(String name, int qty, double price) {
    f.format("%-15.15s %5d %10.2f\n", name, qty, price);
    total += price;
  }
  public void printTotal() {
    f.format("%-15s %5s %10.2f\n", "Tax", "", total*0.06);
    f.format("%-15s %5s %10s\n", "", "", "-----");
    f.format("%-15s %5s %10.2f\n", "Total", "",
      total * 1.06);
  }
  public static void main(String[] args) {
    Receipt receipt = new Receipt();
    receipt.printTitle();
    receipt.print("Jack's Magic Beans", 4, 4.25);
    receipt.print("Princess Peas", 3, 5.1);
    receipt.print("Three Bears Porridge", 1, 14.29);
    receipt.printTotal();
  }
} /* Output:
Item              Qty      Price
----              ---      -----
Jack's Magic Be     4       4.25
Princess Peas       3       5.10
```

517

```
Three Bears Por     1       14.29
Tax                          1.42
                            -----
Total                       25.06
*///:~
```

正如你所见，通过相当简洁的语法，**Formatter**提供了对空格与对齐的强大控制能力。在该程序中，为了恰当地控制间隔，格式化字符串被重复地利用了多遍。

练习4：(3) 修改**Receipt.java**，令所有的宽度都由一个常量来控制。目的是使宽度的改变更容易，只需修改一处的值即可。

13.5.5 Formatter转换

下面的表格包含了最常用的类型转换：

类型转换字符			
d	整数型（十进制）	**e**	浮点数（科学计数）
c	Unicode字符	**x**	整数（十六进制）
b	Boolean值	**h**	散列码（十六进制）
s	String	**%**	字符“%”
f	浮点数（十进制）		

下面的程序演示了这些转换是如何工作的：

518

```
//: strings/Conversion.java
import java.math.*;
import java.util.*;
public class Conversion {
  public static void main(String[] args) {
    Formatter f = new Formatter(System.out);

    char u = 'a';
    System.out.println("u = 'a'");
    f.format("s: %s\n", u);
    // f.format("d: %d\n", u);
    f.format("c: %c\n", u);
    f.format("b: %b\n", u);
    // f.format("f: %f\n", u);
    // f.format("e: %e\n", u);
    // f.format("x: %x\n", u);
    f.format("h: %h\n", u);

    int v = 121;
    System.out.println("v = 121");
    f.format("d: %d\n", v);
    f.format("c: %c\n", v);
    f.format("b: %b\n", v);
    f.format("s: %s\n", v);
    // f.format("f: %f\n", v);
    // f.format("e: %e\n", v);
    f.format("x: %x\n", v);
    f.format("h: %h\n", v);

    BigInteger w = new BigInteger("50000000000000");
    System.out.println(
      "w = new BigInteger(\"50000000000000\")");
    f.format("d: %d\n", w);
    // f.format("c: %c\n", w);
    f.format("b: %b\n", w);
    f.format("s: %s\n", w);
    // f.format("f: %f\n", w);
    // f.format("e: %e\n", w);
    f.format("x: %x\n", w);
```

```
    f.format("h: %h\n", w);

    double x = 179.543;
    System.out.println("x = 179.543");
    // f.format("d: %d\n", x);
    // f.format("c: %c\n", x);
    f.format("b: %b\n", x);
    f.format("s: %s\n", x);
    f.format("f: %f\n", x);
    f.format("e: %e\n", x);
    // f.format("x: %x\n", x);
    f.format("h: %h\n", x);

    Conversion y = new Conversion();
    System.out.println("y = new Conversion()");
    // f.format("d: %d\n", y);
    // f.format("c: %c\n", y);
    f.format("b: %b\n", y);
    f.format("s: %s\n", y);
    // f.format("f: %f\n", y);
    // f.format("e: %e\n", y);
    // f.format("x: %x\n", y);
    f.format("h: %h\n", y);

    boolean z = false;
    System.out.println("z = false");
    // f.format("d: %d\n", z);
    // f.format("c: %c\n", z);
    f.format("b: %b\n", z);
    f.format("s: %s\n", z);
    // f.format("f: %f\n", z);
    // f.format("e: %e\n", z);
    // f.format("x: %x\n", z);
    f.format("h: %h\n", z);
  }
} /* Output: (Sample)
u = 'a'
s: a
c: a
b: true
h: 61
v = 121
d: 121
c: y
b: true
s: 121
x: 79
h: 79
w = new BigInteger("50000000000000")
d: 50000000000000
b: true
s: 50000000000000
x: 2d79883d2000
h: 8842a1a7
x = 179.543
b: true
s: 179.543
f: 179.543000
e: 1.795430e+02
h: 1ef462c
y = new Conversion()
b: true
s: Conversion@9cab16
h: 9cab16
z = false
b: false
s: false
```

519

520

```
h: 4d5
*///:~
```

被注释的代码表示，针对相应类型的变量，这些转换是无效的。如果执行这些转换，则会触发异常。

注意，程序中的每个变量都用到了b转换。虽然它对各种类型都是合法的，但其行为却不一定与你想象的一致。对于**boolean**基本类型或**Boolean**对象，其转换结果是对应的**true**或**false**。但是，对其他类型的参数，只要该参数不为**null**，那转换的结果就永远都是**true**。即使是数字0，转换结果依然为**true**，而这在其他语言中（包括C），往往转换为**false**。所以，将**b**应用于非布尔类型的对象时请格外小心。

还有许多不常用的类型转换与格式修饰符选项，你可以在JDK文档中的**Formatter**类部分找到它们。

练习5：(5) 针对前边表格中的各种基本转化类型，请利用所有可能的格式修饰符，写出一个尽可能复杂的格式化表达式。

13.5.6 String.format()

Java SE5也参考了C中的**sprintf()**方法，以生成格式化的**String**对象。**String.format()**是一个**static**方法，它接受与**Formatter.format()**方法一样的参数，但返回一个**String**对象。当你只需使用**format()**方法一次的时候，**String.format()**用起来很方便。例如：

521

```
//: strings/DatabaseException.java

public class DatabaseException extends Exception {
  public DatabaseException(int transactionID, int queryID,
    String message) {
    super(String.format("(t%d, q%d) %s", transactionID,
        queryID, message));
  }
  public static void main(String[] args) {
    try {
      throw new DatabaseException(3, 7, "Write failed");
    } catch(Exception e) {
      System.out.println(e);
    }
  }
} /* Output:
DatabaseException: (t3, q7) Write failed
*///:~
```

其实在**String.format()**内部，它也是创建一个**Formatter**对象，然后将你传入的参数转给该**Formatter**。不过，与其自己做这些事情，不如使用便捷的**String.format()**方法，何况这样的代码更清晰易读。

一个十六进制转储（dump）工具

在处理二进制文件时，我们经常希望以十六进制的格式看看其内容。现在，我们就将它作为第二个例子。下面的小工具使用了**String.format()**方法，以可读的十六进制格式将字节数组打印出来：

```
//: net/mindview/util/Hex.java
package net.mindview.util;
import java.io.*;

public class Hex {
  public static String format(byte[] data) {
    StringBuilder result = new StringBuilder();
    int n = 0;
    for(byte b : data) {
```

```
      if(n % 16 == 0)
        result.append(String.format("%05X: ", n));
      result.append(String.format("%02X ", b));
      n++;
      if(n % 16 == 0) result.append("\n");
    }
    result.append("\n");
    return result.toString();
  }
  public static void main(String[] args) throws Exception {
    if(args.length == 0)
      // Test by displaying this class file:
      System.out.println(
        format(BinaryFile.read("Hex.class")));
    else
      System.out.println(
        format(BinaryFile.read(new File(args[0]))));
  }
} /* Output: (Sample)
00000: CA FE BA BE 00 00 00 31 00 52 0A 00 05 00 22 07
00010: 00 23 0A 00 02 00 22 08 00 24 07 00 25 0A 00 26
00020: 00 27 0A 00 28 00 29 0A 00 02 00 2A 08 00 2B 0A
00030: 00 2C 00 2D 08 00 2E 0A 00 02 00 2F 09 00 30 00
00040: 31 08 00 32 0A 00 33 00 34 0A 00 15 00 35 0A 00
00050: 36 00 37 07 00 38 0A 00 12 00 39 0A 00 33 00 3A
...
*///:~
```

522

为了打开以及读入二进制文件，我们用到了另一个工具**net.mindview.util.BinaryFile**，在第18章中我会详细地介绍它。这里的**read()**方法将整个文件以**byte**数组的形式返回。

练习6：(2) 创建一个包含**int**、**long**、**float**与**double**元素的类。应用**String.format()**方法为这个类编写**toString()**方法，并证明它能正确工作。

13.6 正则表达式

很久之前，*正则表达式*就已经整合到标准Unix工具集之中，例如sed和awk，以及程序设计语言之中了，例如Python和Perl（有些人认为正是正则表达式促成了Perl的成功）。而在Java中，字符串操作还主要集中于**String**、**StringBuffer**和**StringTokenizer**类。与正则表达式相比较，它们只能提供相当简单的功能。

正则表达式是一种强大而灵活的文本处理工具。使用正则表达式，我们能够以编程的方式，构造复杂的文本模式，并对输入的字符串进行搜索。一旦找到了匹配这些模式的部分，你就能够随心所欲地对它们进行处理。初学正则表达式时，其语法是一个难点，但它确实是一种简洁、动态的语言。正则表达式提供了一种完全通用的方式，能够解决各种字符串处理相关的问题：匹配、选择、编辑以及验证。

523

13.6.1 基础

一般来说，正则表达式就是以某种方式来描述字符串，因此你可以说："如果一个字符串含有这些东西，那么它就是我正在找的东西。"例如，要找一个数字，它可能有一个负号在最前面，那你就写一个负号加上一个问号，就像这样：

```
-?
```

要描述一个整数，你可以说它有一位或多位阿拉伯数字。在正则表达式中，用**\d**表示一位数字。如果在其他语言中使用过正则表达式，那你立刻就能发现Java对反斜线\的不同处理。在其他语言中，\\表示"我想要在正则表达式中插入一个普通的（字面上的）反斜线，请不要给它任何特殊的意义。"而在Java中，\\的意思是"我要插入一个正则表达式的反斜线，所以其

后的字符具有特殊的意义。”例如，如果你想表示一位数字，那么正则表达式应该是**\\d**。如果你想插入一个普通的反斜线，则应该这样\\\\。不过换行和制表符之类的东西只需使用单反斜线：\n\t。

要表示“一个或多个之前的表达式”，应该使用+。所以，如果要表示“可能有一个负号，后面跟着一位或多位数字”，可以这样：

```
-?\\d+
```

应用正则表达式的最简单的途径，就是利用**String**类内建的功能。例如，你可以检查一个**String**是否匹配如上所述的正则表达式：

```
//: strings/IntegerMatch.java

public class IntegerMatch {
  public static void main(String[] args) {
    System.out.println("-1234".matches("-?\\d+"));
    System.out.println("5678".matches("-?\\d+"));
    System.out.println("+911".matches("-?\\d+"));
    System.out.println("+911".matches("(-|\\+)?\\d+"));
  }
} /* Output:
true
true
false
true
*///:~
```

524

前两个字符串满足对应的正则表达式，匹配成功。第三个字符串开头有一个+，它也是一个合法的整数，但与对应的正则表达式却不匹配。因此，我们的正则表达式应该描述为：“可能以一个加号或减号开头”。在正则表达式中，括号有着将表达式分组的效果，而竖直线|则表示或操作。也就是：

```
(-|\\+)?
```

这个正则表达式表示字符串的起始字符可能是一个-或+，或二者皆没有(因为后面跟着?修饰符)。因为字符+在正则表达式中有特殊的意义，所以必须使用\\将其转义，使之成为表达式中的一个普通字符。

String类还自带了一个非常有用的正则表达式工具——**split()**方法，其功能是“将字符串从正则表达式匹配的地方切开。”

```
//: strings/Splitting.java
import java.util.*;

public class Splitting {
  public static String knights =
    "Then, when you have found the shrubbery, you must " +
    "cut down the mightiest tree in the forest... " +
    "with... a herring!";
  public static void split(String regex) {
    System.out.println(
      Arrays.toString(knights.split(regex)));
  }
  public static void main(String[] args) {
    split(" "); // Doesn't have to contain regex chars
    split("\\W+"); // Non-word characters
    split("n\\W+"); // 'n' followed by non-word characters
  }
} /* Output:
[Then,, when, you, have, found, the, shrubbery,, you, must,
cut, down, the, mightiest, tree, in, the, forest...,
with..., a, herring!]
```

```
[Then, when, you, have, found, the, shrubbery, you, must,
cut, down, the, mightiest, tree, in, the, forest, with, a,
herring]
[The, whe, you have found the shrubbery, you must cut dow,
the mightiest tree i, the forest... with... a herring!]
*///:~
```

525

首先看第一个语句，注意这里用的是普通的字符作为正则表达式，其中并不包含任何特殊的字符。因此第一个**split()**只是按空格来划分字符串。

第二个和第三个**split()**都用到了**\W**，它的意思是非单词字符（如果**W**小写，**\w**，则表示一个单词字符）。通过第二个例子可以看到，它将标点字符删除了。第三个**split()**表示“字母**n**后面跟着一个或多个非单词字符。”可以看到，在原始字符串中，与正则表达式匹配的部分，在最终结果中都不存在了。

String.split()还有一个重载的版本，它允许你限制字符串分割的次数。

String类自带的最后一个正则表达式工具是“替换”。你可以只替换正则表达式第一个匹配的子串，或是替换所有匹配的地方。

```
//: strings/Replacing.java
import static net.mindview.util.Print.*;

public class Replacing {
  static String s = Splitting.knights;
  public static void main(String[] args) {
    print(s.replaceFirst("f\\w+", "located"));
    print(s.replaceAll("shrubbery|tree|herring","banana"));
  }
} /* Output:
Then, when you have located the shrubbery, you must cut
down the mightiest tree in the forest... with... a herring!
Then, when you have found the banana, you must cut down the
mightiest banana in the forest... with... a banana!
*///:~
```

第一个表达式要匹配的是，以字母**f**开头，后面跟一个或多个字母（注意这里的**w**是小写的）。并且只替换掉第一个匹配的部分，所以“found”被替换成“located”。

第二个表达式要匹配的是三个单词中的任意一个，因为它们以竖直线分隔表示“或”，并且替换所有匹配的部分。

526

稍后你会看到，**String**之外的正则表达式还有更强大的替换工具，例如，可以通过方法调用执行替换。而且，如果正则表达式不是只使用一次的话，非**String**对象的正则表达式明显具备更佳的性能。

练习7：(5) 请参考**java.util.regex.Pattern**的文档，编写一个正则表达式，检查一个句子是否以大写字母开头，以句号结尾。

练习8：(2) 将字符串**Splitting.knights**在the和you处分割。

练习9：(4) 参考**java.util.regex.Pattern**的文档，用下划线替换**Splitting.knights**中的所有元音字母。

13.6.2 创建正则表达式

我们首先从正则表达式可能存在的构造集中选取一个很有用的子集，以此开始学习正则表达式。正则表达式的完整构造子列表，请参考JDK文档**java.util.regex**包中的**Pattern**类。

字 符	
B	指定字符B
\xhh	十六进制值为oxhh的字符
\uhhhh	十六进制表示为oxhhhh的Unicode字符
\t	制表符Tab
\n	换行符
\r	回车
\f	换页
\e	转义(Escape)

当你学会了使用字符类（character classes）之后，正则表达式的威力才能真正显现出来。以下是一些创建字符类的典型方式，以及一些预定义的类：

字 符 类	
.	任意字符
[abc]	包含a、b和c的任何字符(和 a|b|c作用相同)
[^abc]	除了a、b和c之外的任何字符(否定)
[a-zA-Z]	从a到z或从A到Z的任何字符 (范围)
[abc[hij]]	任意a、b、c、h、i和j字符 (与 a|b|c|h|i|j作用相同) (合并)
[a-z&&[hij]]	任意 h、i或 j (交)
\s	空白符 (空格、tab、换行、换页和回车)
\S	非空白符（[^\s]）
\d	数字[0-9]
\D	非数字[^0-9]
\w	词字符[a-zA-Z0-9]
\W	非词字符[^\w]

527

这里只列出了部分常用的表达式，你应该将JDK文档中**java.util.regex.Pattern**那一页加入浏览器书签中，以便在需要的时候方便查询。

逻辑操作符	
XY	Y跟在X后面
X|Y	X或Y
(X)	捕获组（capturing group）。可以在表达式中用\i引用第i个捕获组

边界匹配符			
^	一行的起始	\B	非词的边界
$	一行的结束	\G	前一个匹配的结束
\b	词的边界		

528

作为演示，下面的每一个正则表达式都能成功匹配字符序列“Rudolph”：

```
//: strings/Rudolph.java

public class Rudolph {
  public static void main(String[] args) {
    for(String pattern : new String[]{ "Rudolph",
      "[rR]udolph", "[rR][aeiou][a-z]ol.*", "R.*" })
      System.out.println("Rudolph".matches(pattern));
  }
} /* Output:
```

```
true .
true
true
true
*///:~
```

当然了，我们的目的并不是编写最难理解的正则表达式，而是尽量编写能够完成任务的、最简单以及最必要的正则表达式。一旦真正开始使用正则表达式了，你就会发现，在编写新的表达式之前，你通常会参考代码中已经用到的正则表达式。

13.6.3 量词

量词描述了一个模式吸收输入文本的方式：

- 贪婪型：量词总是贪婪的，除非有其他的选项被设置。贪婪表达式会为所有可能的模式发现尽可能多的匹配。导致此问题的一个典型理由就是假定我们的模式仅能匹配第一个可能的字符组，如果它是贪婪的，那么它就会继续往下匹配。
- 勉强型：用问号来指定，这个量词匹配满足模式所需的最少字符数。因此也称作懒惰的、最少匹配的、非贪婪的、或不贪婪的。
- 占有型：目前，这种类型的量词只有在Java语言中才可用（在其他语言中不可用），并且也更高级，因此我们大概不会立刻用到它。当正则表达式被应用于字符串时，它会产生相当多的状态，以便在匹配失败时可以回溯。而“占有的”量词并不保存这些中间状态，因此它们可以防止回溯。它们常常用于防止正则表达式失控，因此可以使正则表达式执行起来更有效。

贪婪型	勉强型	占有型	如何匹配
X?	X??	X?+	一个或零个X
X*	X*?	X*+	零个或多个X
X+	X+?	X++	一个或多个X
X{n}	X{n}?	X{n}+	恰好n次X
X{n,}	X{n,}?	X{n,}+	至少n次X
X{n,m}	X{n,m}?	X{n,m}+	X至少n次，且不超过m次

529

应该非常清楚地意识到，表达式X通常必须要用圆括号括起来，以便它能够按照我们期望的效果去执行。例如：

```
abc+
```

看起来它似乎应该匹配1个或多个abc序列，如果我们把它应用于输入字符串abcabcabc，则实际上会获得3个匹配。然而，这个表达式实际上表示的是：匹配ab，后面跟随1个或多个c。要表明匹配1个或多个完整的abc字符串，我们必须这样表示：

```
(abc)+
```

你会发现，在使用正则表达式时很容易混淆，因为它是一种在Java之上的新语言。

CharSequence

接口**CharSequence**从**CharBuffer**、**String**、**StringBuffer**、**StringBuilder**类之中抽象出了字符序列的一般化定义：

```
interface CharSequence {
  charAt(int i);
  length();
  subSequence(int start, int end);
  toString();
}
```

530

因此，这些类都实现了该接口。多数正则表达式操作都接受**CharSequence**类型的参数。

13.6.4 Pattern和Matcher

一般来说，比起功能有限的**String**类，我们更愿意构造功能强大的正则表达式对象。只需导入**java.util.regex**包，然后用**static Pattern.compile()**方法来编译你的正则表达式即可。它会根据你的**String**类型的正则表达式生成一个**Pattern**对象。接下来，把你想要检索的字符串传入**Pattern**对象的**matcher()**方法。**matcher()**方法会生成一个**Matcher**对象，它有很多功能可用（可以参考**java.util.regext.Matcher**的JDK文档）。例如，它的**replaceAll()**方法能将所有匹配的部分都替换成你传入的参数。

作为第一个示例，下面的类可以用来测试正则表达式，看看它们能否匹配一个输入字符串。第一个控制台参数是将要用来搜索匹配的输入字符串，后面的一个或多个参数都是正则表达式，它们将被用来在输入的第一个字符串中查找匹配。在Unix/Linux上，命令行中的正则表达式必须用引号括起。这个程序在测试正则表达式时很有用，特别是当你想验证它们是否具备你所期待的匹配功能的时候。

```
//: strings/TestRegularExpression.java
// Allows you to easily try out regular expressions.
// {Args: abcabcabcdefabc "abc+" "(abc)+" "(abc){2,}" }
import java.util.regex.*;
import static net.mindview.util.Print.*;

public class TestRegularExpression {
  public static void main(String[] args) {
    if(args.length < 2) {
      print("Usage:\njava TestRegularExpression " +
        "characterSequence regularExpression+");
      System.exit(0);
    }
    print("Input: \"" + args[0] + "\"");
    for(String arg : args) {
      print("Regular expression: \"" + arg + "\"");
      Pattern p = Pattern.compile(arg);
      Matcher m = p.matcher(args[0]);
      while(m.find()) {
        print("Match \"" + m.group() + "\" at positions " +
          m.start() + "-" + (m.end() - 1));
      }
    }
  }
} /* Output:
Input: "abcabcabcdefabc"
Regular expression: "abcabcabcdefabc"
Match "abcabcabcdefabc" at positions 0-14
Regular expression: "abc+"
Match "abc" at positions 0-2
Match "abc" at positions 3-5
Match "abc" at positions 6-8
Match "abc" at positions 12-14
Regular expression: "(abc)+"
Match "abcabcabc" at positions 0-8
Match "abc" at positions 12-14
Regular expression: "(abc){2,}"
Match "abcabcabc" at positions 0-8
*///:~
```

531

Pattern对象表示编译后的正则表达式。从这个例子中可以看到，我们使用已编译的**Pattern**对象上的**matcher()**方法，加上一个输入字符串，从而共同构造了一个**Matcher**对象。同时，**Pattern**类还提供了**static**方法：

```
static boolean matches(String regex, CharSequence input)
```

该方法用以检查**regex**是否匹配整个**CharSequence** 类型的**input**参数。编译后的**Pattern**对象还提供了**split()**方法，它从匹配了**regex**的地方分割输入字符串，返回分割后的子字符串**String**数组。

通过调用**Pattern.matcher()**方法，并传入一个字符串参数，我们得到了一个**Matcher**对象。使用**Matcher**上的方法，我们将能够判断各种不同类型的匹配是否成功：

```
boolean matches()
boolean lookingAt()
boolean find()
boolean find(int start)
```

其中的**matches()**方法用来判断整个输入字符串是否匹配正则表达式模式，而**lookingAt()**则用来判断该字符串（不必是整个字符串）的始部分是否能够匹配模式。 532

练习10：(2) 对字符串Java now has regular expressions验证下列正则表达式是否能够发现一个匹配：

```
^Java
\Breg.*
n.w\s+h(a|i)s
s?
s*
s+
s{4}
s{1}.
s{0,3}
```

练习11：(2) 试用正则表达式

```
(?i)((^[aeiou])|(\s+[aeiou]))\w+?[aeiou]\b
```

匹配字符串Arline ate eight apples and one orange while Anita hadn't any。

```
"Arline ate eight apples and one orange while Anita hadn't
any"
```

find()

Matcher.find()方法可用来在**CharSequence**中查找多个匹配。例如：

```
//: strings/Finding.java
import java.util.regex.*;
import static net.mindview.util.Print.*;

public class Finding {
  public static void main(String[] args) {
    Matcher m = Pattern.compile("\\w+")
      .matcher("Evening is full of the linnet's wings");
    while(m.find())
      printnb(m.group() + " ");
    print();
    int i = 0;
    while(m.find(i)) {
      printnb(m.group() + " ");
      i++;
    }
  }
} /* Output:
Evening is full of the linnet s wings
Evening vening ening ning ing ng g is is s full full ull ll
l of of f the the he e linnet linnet innet nnet net et t s
s wings wings ings ngs gs s
*///:~
```

533

模式**\\w+**将字符串划分为单词。**find()**像迭代器那样前向遍历输入字符串。而第二个**find()**能够接收一个整数作为参数，该整数表示字符串中字符的位置，并以其作为搜索的起点。从结果

中可以看出，后一个版本的**find()**方法能根据其参数的值，不断重新设定搜索的起始位置。

组（Groups）

组是用括号划分的正则表达式，可以根据组的编号来引用某个组。组号为0表示整个表达式，组号1表示被第一对括号括起的组，依此类推。因此，在下面这个表达式，

```
A(B(C))D
```

中有三个组：组0是**ABCD**，组1是**BC**，组2是**C**。

Matcher对象提供了一系列方法，用以获取与组相关的信息：**public int groupCount()**返回该匹配器的模式中的分组数目，第0组不包括在内。**public String group()** 返回前一次匹配操作（例如**find()**）的第0组（整个匹配）。**public String group(int i)** 返回在前一次匹配操作期间指定的组号，如果匹配成功，但是指定的组没有匹配输入字符串的任何部分，则将会返回**null**。**public int start(int group)**返回在前一次匹配操作中寻找到的组的起始索引。**public int end(int group)**返回在前一次匹配操作中寻找到的组的最后一个字符索引加一的值。

下面是正则表达式组的例子：

534

```
//: strings/Groups.java
import java.util.regex.*;
import static net.mindview.util.Print.*;
public class Groups {
  static public final String POEM =
    "Twas brillig, and the slithy toves\n" +
    "Did gyre and gimble in the wabe.\n" +
    "All mimsy were the borogoves,\n" +
    "And the mome raths outgrabe.\n\n" +
    "Beware the Jabberwock, my son,\n" +
    "The jaws that bite, the claws that catch.\n" +
    "Beware the Jubjub bird, and shun\n" +
    "The frumious Bandersnatch.";
  public static void main(String[] args) {
    Matcher m =
      Pattern.compile("(?m)(\\S+)\\s+((\\S+)\\s+(\\S+))$")
        .matcher(POEM);
    while(m.find()) {
      for(int j = 0; j <= m.groupCount(); j++)
        printnb("[" + m.group(j) + "]");
      print();
    }
  }
} /* Output:
[the slithy toves][the][slithy toves][slithy][toves]
[in the wabe.][in][the wabe.][the][wabe.]
[were the borogoves,][were][the
borogoves,][the][borogoves,]
[mome raths outgrabe.][mome][raths
outgrabe.][raths][outgrabe.]
[Jabberwock, my son,][Jabberwock,][my son,][my][son,]
[claws that catch.][claws][that catch.][that][catch.]
[bird, and shun][bird,][and shun][and][shun]
[The frumious Bandersnatch.][The][frumious
Bandersnatch.][frumious][Bandersnatch.]
*///:~
```

这首诗来自于Lewis Carroll的《Through the Looking Glass》中的Jabberwocky。可以看到这个正则表达式模式有许多圆括号分组，由任意数目的非空格字符（**\S+**）及随后的任意数目的空格字符（**\s+**）所组成。目的是捕获每行的最后3个词，每行最后以**$**结束。不过，在正常情况下是将**$**与整个输入序列的末端相匹配。所以我们一定要显式地告知正则表达式注意输入序列中的换行符。这可以由序列开头的模式标记**(?m)**来完成（模式标记马上就会介绍）。

535

练习12：(5) 修改**Goups.java**类，找出所有不以大写字母开头的词，不重复地计算其个数。

start() 与 end()

在匹配操作成功之后，**start()**返回先前匹配的起始位置的索引，而**end()**返回所匹配的最后字符的索引加一的值。匹配操作失败之后（或先于一个正在进行的匹配操作去尝试）调用**start()**或 **end()**将会产生**IllegalStateException**。下面的示例还同时展示了**matches()**和**lookingAt()**的用法[⊖]：

```
//: strings/StartEnd.java
import java.util.regex.*;
import static net.mindview.util.Print.*;

public class StartEnd {
  public static String input =
    "As long as there is injustice, whenever a\n" +
    "Targathian baby cries out, wherever a distress\n" +
    "signal sounds among the stars ... We'll be there.\n" +
    "This fine ship, and this fine crew ...\n" +
    "Never give up! Never surrender!";
  private static class Display {
    private boolean regexPrinted = false;
    private String regex;
    Display(String regex) { this.regex = regex; }
    void display(String message) {
      if(!regexPrinted) {
        print(regex);
        regexPrinted = true;
      }
      print(message);
    }
  }
  static void examine(String s, String regex) {
    Display d = new Display(regex);
    Pattern p = Pattern.compile(regex);
    Matcher m = p.matcher(s);
    while(m.find())
      d.display("find() '" + m.group() +
        "' start = "+ m.start() + " end = " + m.end());
    if(m.lookingAt()) // No reset() necessary
      d.display("lookingAt() start = "
        + m.start() + " end = " + m.end());
    if(m.matches()) // No reset() necessary
      d.display("matches() start = "
        + m.start() + " end = " + m.end());
  }
  public static void main(String[] args) {
    for(String in : input.split("\n")) {
      print("input : " + in);
      for(String regex : new String[]{"\\w*ere\\w*",
        "\\w*ever", "T\\w+", "Never.*?!"})
        examine(in, regex);
    }
  }
} /* Output:
input : As long as there is injustice, whenever a
\w*ere\w*
find() 'there' start = 11 end = 16
\w*ever
find() 'whenever' start = 31 end = 39
input : Targathian baby cries out, wherever a distress
\w*ere\w*
find() 'wherever' start = 27 end = 35
```

536

⊖ 引用自电影Galaxy Quest中Taggart司令的一篇演讲。

```
\w*ever
find() 'wherever' start = 27 end = 35
T\w+
find() 'Targathian' start = 0 end = 10
lookingAt() start = 0 end = 10
input : signal sounds among the stars ... We'll be there.
\w*ere\w*
find() 'there' start = 43 end = 48
input : This fine ship, and this fine crew ...
T\w+
find() 'This' start = 0 end = 4
lookingAt() start = 0 end = 4
input : Never give up! Never surrender!
\w*ever
find() 'Never' start = 0 end = 5
find() 'Never' start = 15 end = 20
lookingAt() start = 0 end = 5
Never.*?!
find() 'Never give up!' start = 0 end = 14
find() 'Never surrender!' start = 15 end = 31
lookingAt() start = 0 end = 14
matches() start = 0 end = 31
*///:~
```

537

注意，**find()**可以在输入的任意位置定位正则表达式，而**lookingAt()**和**matches()**只有在正则表达式与输入的最开始处就开始匹配时才会成功。**matches()**只有在整个输入都匹配正则表达式时才会成功，而**lookingAt()**[⊖]只要输入的第一部分匹配就会成功。

练习13：(2) 修改**StartEnd.java**，让它使用**Groups.POEM**为输入，必要时修改正则表达式，使**find()**、**lookingAt()**和**matches()**都有机会匹配成功。

Pattern标记

Pattern类的**compile()**方法还有另一个版本，它接受一个标记参数，以调整匹配的行为：

```
Pattern Pattern.compile(String regex, int flag)
```

其中的**flag**来自以下**Pattern**类中的常量：

编译标记	效　果
Pattern.CANON_EQ	两个字符当且仅当它们的完全规范分解相匹配时，就认为它们是匹配的。例如，如果我们指定这个标记，表达式a\u030A就会匹配字符串?。在默认的情况下，匹配不考虑规范的等价性。
Pattern.CASE_INSENSITIVE(?i)	默认情况下，大小写不敏感的匹配假定只有US-ASCII字符集中的字符才能进行。这个标记允许模式匹配不必考虑大小写（大写或小写）。通过指定**UNICODE_CASE** 标记及结合此标记，基于Unicode的大小写不敏感的匹配就可以开启了
Pattern.COMMENTS(?x)	在这种模式下，空格符将被忽略掉，并且以#开始直到行末的注释也会被忽略掉。通过嵌入的标记表达式也可以开启Unix的行模式
Pattern.DOTALL(?s)	在dotall模式中，表达式“.”匹配所有字符，包括行终结符。默认情况下，“.”表达式不匹配行终结符
Pattern.MULTILINE(?m)	在多行模式下，表达式^和$分别匹配一行的开始和结束。^还匹配输入字符串的开始，而$还匹配输入字符串的结尾。默认情况下，这些表达式仅匹配输入的完整字符串的开始和结束

538

⊖ 我完全不理解设计师怎么会想到这么个方法名。不过可以相信，取出如此不直观的名字的家伙还在Sun工作。而且，不复查代码设计的政策显然还存在于Sun中。请原谅我的讽刺，但是多年来，同样的事情一再发生，这实在令人厌倦。

（续）

编译标记	效 果
Pattern.UNICODE_CASE(?u)	当指定这个标记，并且开启CASE_INSENSITIVE时，大小写不敏感的匹配将按照与Unicode标准相一致的方式进行。默认情况下，大小写不敏感的匹配假定只能在 US-ASCII 字符集中的字符才能进行
Pattern.UNIX_LINES(?d)	在这种模式下，在.、^和$行为中，只识别行终结符\n

539

在这些标记中，**Pattern.CASE_INSENSITIVE**、**Pattern.MULTILINE**以及**Pattern.COMMENTS**（对声明或文档有用）特别有用。请注意，你可以直接在正则表达式中使用其中的大多数标记，只需要将上表中括号括起的字符插入到正则表达式中，你希望它起作用的位置即可。

你还可以通过“或”(|) 操作符组合多个标记的功能：

```
//: strings/ReFlags.java
import java.util.regex.*;

public class ReFlags {
  public static void main(String[] args) {
    Pattern p =  Pattern.compile("^java",
      Pattern.CASE_INSENSITIVE | Pattern.MULTILINE);
    Matcher m = p.matcher(
      "java has regex\nJava has regex\n" +
      "JAVA has pretty good regular expressions\n" +
      "Regular expressions are in Java");
    while(m.find())
      System.out.println(m.group());
  }
} /* Output:
java
Java
JAVA
*///:~
```

在这个例子中，我们创建了一个模式，它将匹配所有以“java”、“Java”和“JAVA”等开头的行，并且是在设置了多行标记的状态下，对每一个行（从字符序列的第一个字符开始，至每一个行终结符）都进行匹配。注意，**group()**方法只返回已匹配的部分。

13.6.5 split()

split()方法将输入字符串断开成字符串对象数组，断开边界由下列正则表达式确定：

```
String[] split(CharSequence input)
String[] split(CharSequence input, int limit)
```

540

这是一个快速而方便的方法，可以按照通用边界断开输入文本：

```
//: strings/SplitDemo.java
import java.util.regex.*;
import java.util.*;
import static net.mindview.util.Print.*;

public class SplitDemo {
  public static void main(String[] args) {
    String input =
      "This!!unusual use!!of exclamation!!points";
    print(Arrays.toString(
      Pattern.compile("!!").split(input)));
    // Only do the first three:
    print(Arrays.toString(
      Pattern.compile("!!").split(input, 3)));
  }
```

```
} /* Output:
[This, unusual use, of exclamation, points]
[This, unusual use, of exclamation!!points]
*///:~
```

第二种形式的**split()**方法可以限制将输入分割成字符串的数量。

练习14：(1) 用**String.split()**重写**SplitDemo**。

13.6.6 替换操作

正则表达式特别便于替换文本，它提供了许多方法：**replaceFirst(String replacement)** 以参数字符串**replacement**替换掉第一个匹配成功的部分。**replaceAll(String replacement)** 以参数字符串**replacement**替换所有匹配成功的部分。**appendReplacement(StringBuffer sbuf, String replacement)** 执行渐进式的替换，而不是像**replaceFirst()**和**replaceAll()**那样只替换第一个匹配或全部匹配。这是一个非常重要的方法。它允许你调用其他方法来生成或处理**replacement**（**replaceFirst()**和**replaceAll()**则只能使用一个固定的字符串），使你能够以编程的方式将目标分割成组，从而具备更强大的替换功能。**appendTail(StringBuffer sbuf)**，在执行了一次或多次**appendReplacement()**之后，调用此方法可以将输入字符串余下的部分复制到**sbuf**中。

541

下面的程序演示了如何使用这些替换方法。开头部分注释掉的文本，就是正则表达式要处理的输入字符串。

```
//: strings/TheReplacements.java
import java.util.regex.*;
import net.mindview.util.*;
import static net.mindview.util.Print.*;

/*! Here's a block of text to use as input to
    the regular expression matcher. Note that we'll
    first extract the block of text by looking for
    the special delimiters, then process the
    extracted block. !*/

public class TheReplacements {
  public static void main(String[] args) throws Exception {
    String s = TextFile.read("TheReplacements.java");
    // Match the specially commented block of text above:
    Matcher mInput =
      Pattern.compile("/\\*!(.*)!\\*/", Pattern.DOTALL)
        .matcher(s);
    if(mInput.find())
      s = mInput.group(1); // Captured by parentheses
    // Replace two or more spaces with a single space:
    s = s.replaceAll(" {2,}", " ");
    // Replace one or more spaces at the beginning of each
    // line with no spaces. Must enable MULTILINE mode:
    s = s.replaceAll("(?m)^ +", "");
    print(s);
    s = s.replaceFirst("[aeiou]", "(VOWEL1)");
    StringBuffer sbuf = new StringBuffer();
    Pattern p = Pattern.compile("[aeiou]");
    Matcher m = p.matcher(s);
    // Process the find information as you
    // perform the replacements:
    while(m.find())
      m.appendReplacement(sbuf, m.group().toUpperCase());
    // Put in the remainder of the text:
    m.appendTail(sbuf);
    print(sbuf);
  }
} /* Output:
Here's a block of text to use as input to
```

542

```
the regular expression matcher. Note that we'll
first extract the block of text by looking for
the special delimiters, then process the
extracted block.
H(VOWEL1)rE's A blOck Of tExt tO UsE As InpUt tO
thE rEgUlAr ExprEssIOn mAtchEr. NOtE thAt wE'll
fIrst ExtrAct thE blOck Of tExt by lOOkIng fOr
thE spEcIAl dElImItErs, thEn prOcEss thE
ExtrActEd blOck.
*///:~
```

此处使用**TextFile**类打开并读入文件，该类在**net.mindview.util**工具包中（在第18章中对其代码有详细介绍）。**static read()**方法读入整个文件，将其内容作为**String**对象返回。**mInput**用以匹配在/*!和!*/之间的所有文字（注意分组的括号）。接下来，将存在两个或两个以上空格的地方，缩减为一个空格，并且删除每行开头部分的所有空格（为了使每一行都达到这个效果，而不仅仅只是删除文本开头部分的空格，这里特意打开了多行状态）。这两个替换操作所使用的**replaceAll()**是**String**对象自带的方法，在这里，使用此方法更方便。注意，因为这两个替换操作都只使用了一次**replaceAll()**，所以，与其编译为**Pattern**，不如直接使用**String**的**replaceAll()**方法，而且开销也更小些。

replaceFirst()只对找到的第一个匹配进行替换。此外，**replaceFirst()**和**repalceAll()**方法用来替换的只是普通的字符串，所以，如果想对这些替换字符串执行某些特殊处理，这两个方法是无法胜任的。如果你想要那么做，就应该使用**appendReplacement()**方法。该方法允许你在执行替换的过程中，操作用来替换的字符串。在这个例子中，先构造了**sbuf**用来保存最终结果，然后用**group()**选择一个组，并对其进行处理，将正则表达式找到的元音字母转换成大写字母。一般情况下，你应该遍历执行所有的替换操作，然后再调用**appendTail()**方法，但是，如果你想模拟**replaceFirst()**（或替换n次）的行为，那就只需执行一次替换，然后调用**appendTail()**方法，将剩余未处理的部分存入**sbuf**即可。 543

同时，**appendRepelacement()**方法还允许你通过$g直接找到匹配的某个组，这里的g就是组号。然而，它只能应付一些简单的处理，无法实现类似前面这个例子中的功能。

13.6.7 reset()

通过**reset()**方法，可以将现有的**Matcher**对象应用于一个新的字符序列：

```
//: strings/Resetting.java
import java.util.regex.*;

public class Resetting {
  public static void main(String[] args) throws Exception {
    Matcher m = Pattern.compile("[frb][aiu][gx]")
      .matcher("fix the rug with bags");
    while(m.find())
      System.out.print(m.group() + " ");
    System.out.println();
    m.reset("fix the rig with rags");
    while(m.find())
      System.out.print(m.group() + " ");
  }
} /* Output:
fix rug bag
fix rig rag
*///:~
```

使用不带参数的**reset()**方法，可以将**Matcher**对象重新设置到当前字符序列的起始位置。

13.6.8 正则表达式与Java I/O

到目前为止，我们看到的例子都是将正则表达式应用于静态的字符串。下面的例子将向你

演示，如何应用正则表达式在一个文件中进行搜索匹配操作。**JGrep.java**的灵感源自于Unix上的grep。它有两个参数：文件名以及要匹配的正则表达式。输出的是有匹配的部分以及匹配部分在行中的位置。

544

```
//: strings/JGrep.java
// A very simple version of the "grep" program.
// {Args: JGrep.java "\\b[Ssct]\\w+"}
import java.util.regex.*;
import net.mindview.util.*;

public class JGrep {
  public static void main(String[] args) throws Exception {
    if(args.length < 2) {
      System.out.println("Usage: java JGrep file regex");
      System.exit(0);
    }
    Pattern p = Pattern.compile(args[1]);
    // Iterate through the lines of the input file:
    int index = 0;
    Matcher m = p.matcher("");
    for(String line : new TextFile(args[0])) {
      m.reset(line);
      while(m.find())
        System.out.println(index++ + ": " +
          m.group() + ": " + m.start());
    }
  }
} /* Output: (Sample)
0: strings: 4
1: simple: 10
2: the: 28
3: Ssct: 26
4: class: 7
5: static: 9
6: String: 26
7: throws: 41
8: System: 6
9: System: 6
10: compile: 24
11: through: 15
12: the: 23
13: the: 36
14: String: 8
15: System: 8
16: start: 31
*///:~
```

通过**net.mindview.util.TextFile**对象将文件打开（在第18章中有详细的介绍），读入所有的行后，并存储在一个**ArrayList**中。因此，可以用循环来迭代遍历**TextFile**对象中的所有行。虽然也可以在**for**循环内部创建新的**Matcher**对象，但是，在循环外创建一个空的**Matcher**对象，然后用**reset()**方法每次为**Matcher**加载一行输入，这种处理会有一定的性能优化。最后用**find()**搜索结果。这里读入的测试参数是**JGrep.java**文件，然后搜索以**[Ssct]**开头的单词。

545

如果想要更深入的学习正则表达式，你可以阅读Jeffrey E. F. Friedl的《精通正则表达式（第2版）》。网络上也有很多正则表达式的介绍，你还可以从Perl和Python等其他语言的文档中找到有用的信息。

练习15：(5) 修改**JGrep.java**类，令其能够接受模式标志参数（例如**Pattern.CASE_INSENSITIVE**，**Pattern.MULTILINE**）。

练习16：(5) 修改**JGrep.java**类，令其能够接受一个目录或文件为参数（如果传入的是目录，

就搜索目录中的所有文件)。提示：可以用下面的方法获得所有文件的名字列表：

```
File[] files = new File(".").listFiles();
```

练习17：(8) 编写一个程序，读取一个Java源代码文件（可以通过控制台参数提供文件名），打印出所有注释。

练习18：(8) 编写一个程序，读取一个Java源代码文件（可以通过控制台参数提供文件名），打印出代码中所有的普通字符串。

练习19：(8) 在前两个练习的基础上，编写一个程序，输出Java源代码中用到的所有类的名字。

13.7 扫描输入

到目前为止，从文件或标准输入读取数据还是一件相当痛苦的事情。一般的解决之道就是读入一行文本，对其进行分词，然后使用**Integer**、**Double**等类的各种解析方法来解析数据：

```
//: strings/SimpleRead.java
import java.io.*;
public class SimpleRead {
  public static BufferedReader input = new BufferedReader(
    new StringReader("Sir Robin of Camelot\n22 1.61803"));
  public static void main(String[] args) {
    try {
      System.out.println("What is your name?");
      String name = input.readLine();
      System.out.println(name);
      System.out.println(
        "How old are you? What is your favorite double?");
      System.out.println("(input: <age> <double>)");
      String numbers = input.readLine();
      System.out.println(numbers);
      String[] numArray = numbers.split(" ");
      int age = Integer.parseInt(numArray[0]);
      double favorite = Double.parseDouble(numArray[1]);
      System.out.format("Hi %s.\n", name);
      System.out.format("In 5 years you will be %d.\n",
        age + 5);
      System.out.format("My favorite double is %f.",
        favorite / 2);
    } catch(IOException e) {
      System.err.println("I/O exception");
    }
  }
} /* Output:
What is your name?
Sir Robin of Camelot
How old are you? What is your favorite double?
(input: <age> <double>)
22 1.61803
Hi Sir Robin of Camelot.
In 5 years you will be 27.
My favorite double is 0.809015.
*///:~
```

546

input元素使用的类来自**java.io**，在第18章中我们会正式介绍这个包中的内容。**StringReader**将**String**转化为可读的流对象，然后用这个对象来构造**BufferReader**对象，因为我们要使用**BufferReader**的**readLine()**方法。最终，我们可以使用**input**对象一次读取一行文本，就像是从控制台读入标准输入一样。

547

readLine()方法将一行输入转为**String**对象。如果每一行数据正好对应一个输入值，那这个

方法也还可行。但是，如果两个输入值在同一行中，事情就不好办了，我们必须分解这个行，才能分别翻译所需的输入值。在这个例子中，分解的操作发生在创建**numArray**时，不过要注意，**split()**方法是J2SE1.4中的方法，所以在这之前，你还必须做些别的准备。

终于，Java SE5新增了**Scanner**类，它可以大大减轻扫描输入的工作负担：

```
//: strings/BetterRead.java
import java.util.*;

public class BetterRead {
  public static void main(String[] args) {
    Scanner stdin = new Scanner(SimpleRead.input);
    System.out.println("What is your name?");
    String name = stdin.nextLine();
    System.out.println(name);
    System.out.println(
      "How old are you? What is your favorite double?");
    System.out.println("(input: <age> <double>)");
    int age = stdin.nextInt();
    double favorite = stdin.nextDouble();
    System.out.println(age);
    System.out.println(favorite);
    System.out.format("Hi %s.\n", name);
    System.out.format("In 5 years you will be %d.\n",
      age + 5);
    System.out.format("My favorite double is %f.",
      favorite / 2);
  }
} /* Output:
What is your name?
Sir Robin of Camelot
How old are you? What is your favorite double?
(input: <age> <double>)
22
1.61803
Hi Sir Robin of Camelot.
In 5 years you will be 27.
My favorite double is 0.809015.
*///:~
```

548

Scanner的构造器可以接受任何类型的输入对象，包括**File**对象（同样，我们将在第18章中详细介绍**File**类）、**InputStream**、**String**或者像此例中的**Readable**对象。**Readable**是Java SE5中新加入的一个接口，表示“具有**read()**方法的某种东西”。前一个例子中的**BufferedReader**也归于这一类。有了**Scanner**，所有的输入、分词以及翻译的操作都隐藏在不同类型的**next**方法中。普通的**next()**方法返回下一个**String**。所有的基本类型（除**char**之外）都有对应的**next**方法，包括**BigDecimal**和**BigInteger**。所有的**next**方法，只有在找到一个完整的分词之后才会返回。**Scanner**还有相应的**hasNext**方法，用以判断下一个输入分词是否所需的类型。

在前面的两个例子中，一个有趣的区别是，**BetterRead.java**没有针对**IOException**添加**try**区块。因为，**Scanner**有一个假设，在输入结束时会抛出**IOException**，所以**Scanner**会把**IOException**吞掉。不过，通过**ioException()**方法，你可以找到最近发生的异常，因此，你可以在必要时检查它。

练习20：(2) 编写一个包含**int**、**long**、**float**、**double**和**String**属性的类。为它编写一个构造器，接收一个**String**参数。然后扫描该字符串，为各个属性赋值。再添加一个**toString()**方法，用来演示你的类是否工作正确。

13.7.1 Scanner定界符

在默认的情况下，**Scanner**根据空白字符对输入进行分词，但是你可以用正则表达式指定自

己所需的定界符：

```
//: strings/ScannerDelimiter.java
import java.util.*;

public class ScannerDelimiter {
  public static void main(String[] args) {
    Scanner scanner = new Scanner("12, 42, 78, 99, 42");
    scanner.useDelimiter("\\s*,\\s*");
    while(scanner.hasNextInt())
      System.out.println(scanner.nextInt());
  }
} /* Output:
12
42
78
99
42
*///:~
```

549

这个例子使用逗号（包括逗号前后任意的空白字符）作为定界符，同样的技术也可以用来读取逗号分隔的文件。我们可以用**useDelimiter()**来设置定界符，同时，还有一个**delimiter()**方法，用来返回当前正在作为定界符使用的**Pattern**对象。

13.7.2 用正则表达式扫描

除了能够扫描基本类型之外，你还可以使用自定义的正则表达式进行扫描，这在扫描复杂数据的时候非常有用。下面的例子将扫描一个防火墙日志文件中记录的威胁数据：

```
//: strings/ThreatAnalyzer.java
import java.util.regex.*;
import java.util.*;

public class ThreatAnalyzer {
  static String threatData =
    "58.27.82.161@02/10/2005\n" +
    "204.45.234.40@02/11/2005\n" +
    "58.27.82.161@02/11/2005\n" +
    "58.27.82.161@02/12/2005\n" +
    "58.27.82.161@02/12/2005\n" +
    "[Next log section with different data format]";
  public static void main(String[] args) {
    Scanner scanner = new Scanner(threatData);
    String pattern = "(\\d+[.]\\d+[.]\\d+[.]\\d+)@" +
      "(\\d{2}/\\d{2}/\\d{4})";
    while(scanner.hasNext(pattern)) {
      scanner.next(pattern);
      MatchResult match = scanner.match();
      String ip = match.group(1);
      String date = match.group(2);
      System.out.format("Threat on %s from %s\n", date,ip);
    }
  }
} /* Output:
Threat on 02/10/2005 from 58.27.82.161
Threat on 02/11/2005 from 204.45.234.40
Threat on 02/11/2005 from 58.27.82.161
Threat on 02/12/2005 from 58.27.82.161
Threat on 02/12/2005 from 58.27.82.161
*///:~
```

550

当**next()**方法配合指定的正则表达式使用时，将找到下一个匹配该模式的输入部分，调用**match()**方法就可以获得匹配的结果。如上所示，它的工作方式与之前看到正则表达式匹配相似。在配合正则表达式使用扫描时，有一点需要注意：它仅仅针对下一个输入分词进行匹配，如果

你的正则表达式中含有定界符，那永远都不可能匹配成功。

13.8 StringTokenizer

在Java引入正则表达式（J2SE1.4）和**Scanner**类（Java SE5）之前，分割字符串的唯一方法是使用**StringTokenizer**来分词。不过，现在有了正则表达式和**Scanner**，我们可以使用更加简单、更加简洁的方式来完成同样的工作了。下面的例子中，我们将**StringTokenizer**与另两种技术做了一个比较：

```
//: strings/ReplacingStringTokenizer.java
import java.util.*;

public class ReplacingStringTokenizer {
  public static void main(String[] args) {
    String input = "But I'm not dead yet! I feel happy!";
    StringTokenizer stoke = new StringTokenizer(input);
    while(stoke.hasMoreElements())
      System.out.print(stoke.nextToken() + " ");
    System.out.println();
    System.out.println(Arrays.toString(input.split(" ")));
    Scanner scanner = new Scanner(input);
    while(scanner.hasNext())
      System.out.print(scanner.next() + " ");
  }
} /* Output:
But I'm not dead yet! I feel happy!
[But, I'm, not, dead, yet!, I, feel, happy!]
But I'm not dead yet! I feel happy!
*///:~
```

551

使用正则表达式或**Scanner**对象，我们能够以更加复杂的模式来分割一个字符串，而这对于**StringTokenizer**来说就很困难了。基本上，我们可以放心的说，**StringTokenizer**已经可以废弃不用了。

13.9 总结

过去，Java对字符串操作的支持相当不完善。不过随着近几个版本的升级，我们可以看到，Java已经从其他语言中吸取了许多成熟的经验。到目前为止，它对字符串操作的支持已经很完善了。不过，有时你还要在细节上注意效率的问题，例如恰当地使用**StringBuilder**等。

所选习题的答案都可以在名为The Thinking in Java Annotated Solution Guide的电子文档中找到，读者可以从www.MindView.net处购买此文档。

552